

Aula 06 - Estruturas Condicionais, Repetição e Controle em IoT

Descrição: Implementação e validação de estruturas de decisão (if/else, switch/case) e laços (for, while) no simulador virtual de IoT.

1. Abertura e Apresentação do Problema

O Caso AgroTech: O Resgate da Estufa Inteligente

A empresa de agrotecnologia *AgroSmart* contratou sua equipe para reformular o firmware do controlador de uma estufa automatizada. O dispositivo atual apresenta graves falhas operacionais:

1. **Travamentos Frequentes:** O sistema para de responder e perde a conexão Wi-Fi porque o código antigo utiliza pausas completas do processador para contar tempo.
2. **Falhas de Segurança:** Quando a temperatura atinge níveis críticos, o sistema demora a reagir porque fica preso dentro de loops de leitura secundários.
3. **Corrupção de Registradores:** Ao tentar ligar a bomba de água, o código antigo reescreve todo o byte de memória, alterando acidentalmente as configurações dos sensores de umidade .

Sua Missão Hoje: Até o final desta aula, sua equipe deverá projetar, programar e depurar um novo firmware seguro em C para a estufa no simulador virtual (<https://www.onlinegdb.com/>). Para construir essa solução, você aprenderá e aplicará as ferramentas técnicas a seguir .

2. Ferramentas Técnicas I: Manipulação de Bits (Bitwise) e Registradores

Para controlar o hardware da estufa sem gastar memória excessiva nem corromper dados, utilizaremos operações no nível de bit .

2.1 Aritmética de Bits e Operadores em C

Os operadores bit-a-bit realizam cálculos diretamente na representação binária (0s e 1s) dos números.

- **AND (&) — Máscara de Leitura:** O bit resultante será 1 apenas se ambos os bits comparados forem 1.
 - *Aplicação na Estufa:* Usado para **isolar** um bit específico e verificar se um pino de hardware ou flag de sensor está ativo.
- **OR (|) — Ativação de Bits (Set):** O bit resultante será 1 se pelo menos um dos bits for 1.
 - *Aplicação na Estufa:* A ferramenta ideal para **"ligar"** um bit específico (ex: ativar a bomba de água) sem alterar os outros bits do registrador.
- **XOR (^) — Inversão de Estado (Toggle):** O bit resultante será 1 se os bits comparados forem diferentes.
 - *Aplicação na Estufa:* Usado para **alternar** o estado de um bit (se é 0 vira 1, se é 1 vira 0). Ótimo para fazer um LED de status piscar.
- **NOT (~) — Inversão Total:** Operador unário que inverte todos os bits do valor (transforma 0 em 1 e 1 em 0).
- **Deslocamentos (<< e >>):** Movem os bits de um valor \$N\$ posições para a esquerda ou para a direita.
 - Deslocar 1 posição para a esquerda (<< 1) equivale a **multiplicar por 2**.
 - Deslocar 1 posição para a direita (>> 1) equivale a uma **divisão inteira por 2**.

2.2 Mapeamento de Memória e Registradores

Por que alteramos bits individuais em vez de mudar bytes inteiros?

Em microcontroladores e sistemas de IoT, os periféricos de hardware são mapeados diretamente no mesmo espaço da memória RAM. Um único endereço de memória (um byte de 8 bits) gerencia vários componentes ao mesmo tempo (por exemplo: Bit 0 = Umidade, Bit 1 = Bomba, Bit 2 = Alerta Térmico) .

Se alterássemos o byte inteiro para ligar a bomba, poderíamos acidentalmente desligar a leitura do sensor de umidade. O uso de **máscaras de bits** garante que alteramos apenas o recurso desejado, preservando a integridade do sistema.

3. Ferramentas Técnicas II: Lógica de Decisão e Estruturação de Código

3.1 Tomada de Decisão Booleana (if, else if, else)

O controle de fluxo permite ao programa escolher caminhos com base em condições.

- **if / else:** Executa um bloco de código se a condição for verdadeira; o else trata o caminho alternativo.
- **else if:** Encadeia verificações para tratar múltiplos caminhos possíveis.
- **Avaliação Booleana em C:** A linguagem C considera o valor **Zero como FALSO** e **qualquer valor Diferente de Zero como VERDADEIRO**.

3.2 Combinação de Operadores Lógicos e Bitwise

Podemos testar condições lógicas (&&, ||, !) juntamente com operações em bits (&, |).

- **Atenção à Precedência:** Os operadores bitwise (&, |) possuem **maior precedência** (são processados antes) que os operadores lógicos (&&, ||). Isso permite filtrar um bit com uma máscara & e checar o resultado dentro de uma estrutura if na mesma linha.

3.3 Código Limpo: Cláusulas de Guarda (*Guard Clauses*)

O acúmulo de múltiplos ifs aninhados (um dentro do outro) prejudica a leitura e a manutenção do código.

- **Técnica da Cláusula de Guarda:** Trata e interrompe erros precocemente. Em vez de envolver todo o código do programa em um bloco if gigante, testamos a condição de emergência logo nas primeiras linhas e interrompemos o fluxo com return ou alteração direta de estado.
- *Uso na Estufa:* Se o sensor indicar superaquecimento, a cláusula de guarda é acionada na hora, impedindo que o código tente irrigar a planta antes de tratar o risco térmico.

3.4 Decisão Múltipla Otimizada (switch / case)

O comando switch é uma estrutura de seleção usada para comparar uma única variável com múltiplos valores constantes preestabelecidos.

- **switch e case:** A variável (de tipo inteiro int ou caractere char) é comparada com os valores de cada case.
- **break:** Interrompe a execução assim que um case correspondente é executado. Sem o break, o programa continuará executando os comandos dos case abaixo.
- **default:** O caminho padrão executado se nenhuma opção for satisfeita.
- **Desempenho Otimizado (Tabelas de Desvio / *Jump Tables*):** Diferente de uma longa cadeia de else if, o compilador converte o switch em uma tabela de saltos em memória, tornando a transição de estados instantânea.
- **Modelagem de Máquina de Estados:** É a ferramenta perfeita para gerenciar modos de operação.

- Exemplo de Estados: MONITORAMENTO, IRRIGAÇÃO e ALERTA .
-

4. Ferramentas Técnicas III: Laços de Repetição, Confiabilidade e Tempo

4.1 Laço Contado (for)

Concentra a inicialização, condição de parada e incremento em uma única linha.

- **Anatomia:** for (inicialização; condição_de_parada; incremento) .
- **Aplicações em IoT:**
 - **Varredura e Média de Sensores:** Executa um número fixo de leituras de um sensor para calcular a média aritmética e descartar ruídos elétricos.
 - **Buffers de Comunicação:** Percorre arrays de memória para organizar dados antes da transmissão.

4.2 Laços Condicionais Baseados em Estado (while e do-while)

Usados quando não sabemos previamente quantas vezes a repetição acontecerá.

- **while (Verificação no Início):** Testa a condição antes de rodar o bloco. Se a condição for falsa na chegada, o código interno **nunca é executado**. Ideal para rotinas de espera por sensores.
- **do-while (Verificação no Fim):** Executa o bloco de código primeiro e só depois testa a condição. Garante a execução **pelo menos uma vez**. Perfeito para rotinas de calibração de sensores ou tentativas de conexão ao Wi-Fi.
- **Flags de Controle:** Variáveis que sinalizam o momento de interromper um laço para evitar que ele fique preso para sempre.

4.3 Segurança e o Arquétipo do Super Loop

- **Loops Infinitos Indesejados:** Ocorrem quando o programador esquece de atualizar a variável de controle dentro do laço, travando o processador .
- **Super Loop Proposital (while(1) ou for(;;)):** A espinha dorsal da programação de firmware. Dispositivos de IoT não possuem sistema operacional para "fechar o programa"; eles devem rodar continuamente enquanto houver energia .
- **O Perigo do Bloqueio de Código (Blocking Code):** Se o firmware ficar preso dentro de uma tarefa longa, o envio de dados pela rede Wi-Fi e os alarmes de emergência deixarão de funcionar .

4.4 Depuração Embarcada (Debugging)

Como diagnosticar falhas no simulador:

- **Mensagens de Rastreo (Tracing):** Utilização da função printf() para exibir via console/saída serial os valores das variáveis em tempo real.
- **Breakpoints e Janela de Inspeção (Watches):** Pausar o programa em linhas estratégicas para examinar se as *flags* de estado foram alteradas corretamente .
- **Execução Passo a Passo:** Avançar instrução por instrução para identificar o ponto exato onde a lógica falha.

4.5 Gestão do Tempo: Atrasos Bloqueantes vs. Não-Bloqueantes

- **O Problema do delay() Bloqueante:** A função delay() ou sleep() coloca o processador em um estado de espera passiva. Durante essa pausa, o processador fica totalmente "cego": não lê sensores de emergência

e consome energia da bateria inutilmente .

- **Abordagem Não-Bloqueante com Temporizadores (<time.h>):** Em vez de mandar a CPU parar, consultamos o relógio do sistema (clock() ou time()). O programa armazena a hora de início e, a cada passagem pelo Super Loop, calcula a diferença de tempo (difftime). Se o intervalo necessário não passou, o processador fica livre para executar outras tarefas com agilidade.
-

5. Atividade Prática Integrada: Construção do Firmware

Com base nas ferramentas aprendidas, monte o novo firmware da **Estufa Inteligente** no simulador virtual.

Requisitos Técnicos do Projeto

1. **Estado do Hardware (Bitwise):** Declare uma variável de 8 bits do tipo unsigned char status_estufa para gerenciar os dispositivos:
 - **Bit 0:** Sensor de Umidade (0: Seco / 1: Úmido).
 - **Bit 1:** Bomba d'Água (0: Desligada / 1: Ligada).
 - **Bit 2:** Alerta de Temperatura (0: OK / 1: Crítico).
 2. **Segurança (Cláusula de Guarda):** No início do ciclo, verifique se o bit de "Alerta de Temperatura" está ativo. Se estiver, force imediatamente a transição para o estado de ALERTA e desligue a bomba de água.
 3. **Lógica de Controle (Máquina de Estados):** Implemente um switch / case com três estados:
 - **MONITORAMENTO:** Coleta dados do solo.
 - **IRRIGAÇÃO:** Ativa o Bit 1 (Bomba) via operador | se o solo estiver seco (Bit 0 = 0).
 - **ALERTA:** Mantém atuadores em modo de segurança e exibe mensagem de emergência.
 4. **Super Loop e Tempo Não-Bloqueante:**
 - O sistema deve rodar dentro de uma estrutura while(1).
 - **É proibido usar** delay(). Utilize a biblioteca <time.h> para imprimir o status do sistema no console **apenas a cada 3 segundos**.
 5. **Sinalização Visual:** Alterne (toggle) o bit de um LED de status a cada ciclo de leitura utilizando o operador XOR (^).
-

6. Apresentação das Soluções e Encerramento

Ao término do tempo de desenvolvimento, cada grupo apresentará brevemente sua solução para a turma.

Roteiro de Apresentação (5 minutos por grupo):

1. **Demonstração do Simulador:** Exibir o funcionamento da máquina de estados no console e o comportamento do tempo não-bloqueante.
2. **Exibição do Código Limpo:** Demonstrar como a Cláusula de Guarda evitou que o sistema irrigasse em estado de superaquecimento.
3. **Defesa do Bitwise:** Explicar qual máscara de bit foi utilizada para acionar a bomba de água sem alterar o sensor de umidade.
4. **Relato de Bug:** Compartilhar um erro de lógica enfrentado durante o desenvolvimento e como ele foi resolvido usando o printf() ou ferramentas do depurador.